

MIAE 215 · Mini-Course Lesson 2

Anatomy of C++ Programs & First Hello World

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Source: [Lesson 2 Web Page](#) | Video: [lesson2_helloworld.mp4](#)

Table of Contents

1. [The Minimal C++ Program](#)
 2. [Line-by-Line Code Decomposition](#)
 3. [Stream Output Mechanics: `cout` and Escape Sequences](#)
 4. [Preventing Instant Window Closure: `getchar\(\)`](#)
 5. [Teacher Source Code: `lesson2.cpp`](#)
-

1. The Minimal C++ Program

In this lesson, Prof. Gordon introduces the fundamental structure of every C++ executable. Below is the complete code from `lesson2.cpp`:

```
#include <iostream>
#include <cstdio>

using namespace std;

int main()
{
    cout << "Hello World!
";

    cout << "
Press enter to continue.";
    getchar();

    return 0;
}
```

2. Line-by-Line Code Decomposition

#include <iostream>	← Preprocessor: Loads streams
#include <cstdio>	← Standard C I/O (<code>getchar</code>)
using namespace std;	← Exposes <code>std</code> namespace

```

| int main()           ← Program Entry Point |
| { |
|     cout << "Hello World!" |
"; ← Output to console |
|     getchar();         ← Pauses console |
|     return 0;          ← Exit code 0 (clean) |
| } |

```

1. `#include <iostream>` : Instructs the preprocessor to pull in declarations for standard stream I/O objects (`cout` , `cin`).
2. `#include <cstdio>` : Needed for `getchar()` . In older compilers, omitting this can cause compilation failure.
3. `using namespace std;` : Allows writing `cout` and `cin` directly instead of qualifying every entity with `std::cout` and `std::cin` .
4. `int main()` : The mandatory entry point. Every C++ program must contain exactly one `main()` function. Execution begins here sequentially.
5. `return 0;` : Returns status code `0` to the operating system, signaling successful completion without runtime faults.

3. Stream Output Mechanics: `cout` and Escape Sequences

The `cout` stream object (pronounced "see-out") sends characters to standard output using the **stream insertion operator** (`<<`):

```
cout << "Measurement 1: " << 45.8 << " cm
";
```

Escape Sequences

- `\n` : Newline character. Moves cursor to the start of the next line.
- `\t` : Tab character. Useful for aligning numeric data tables.
- `\\` : Prints a literal backslash.
- `\"` : Prints double quotation marks inside a string.

4. Preventing Instant Window Closure: `getchar()`

When you run a console executable on Windows outside an IDE (or in certain Code::Blocks environments), Windows automatically closes the command prompt window the microsecond `return 0;` executes.

- Calling `getchar()` halts execution and waits until the user presses the **Enter** key.
- This keeps the console open so the user can inspect output data.

5. Teacher Source Code: `lesson2.cpp`

The raw source file is located at [./lesson2.cpp](#) . Build and run it in Code::Blocks to verify output.